인공지능을 위한
PYTHON

PYTHON 자료형 - SET

◆ 집합 자료형

변경집합	set타입	class ‘set’	다양한 데이터
불변집합	frozenset타입	class ‘frozenset’	다양한 데이터

◆ Set 자료형

- 파이썬 2.3부터 지원된 자료형
- 집합 데이터 관리
- **중복 불허, 순서 없음**

변수명=set(데이터)

◆ Set 자료형

❖ 생성

```
numSet1 = { 3, 5, 7, 2, 4 , 5, 7 }
print(f"numSet1 => {type(numSet1)}, {numSet1}")

numSet1 = {"1", 3, 5, (1,3)}
print(f"numSet1 => {type(numSet1)}, {numSet1}")

#numSet1 = {"1", 3, 5, [1,3]}    # 수정가능 타입은 사용 불가
print(f"numSet1 => {type(numSet1)}, {numSet1}")

numSet1 = {3, 5, 7, 3, 1, 2, 19, 21, 3, 5}
print(f"numSet1 => {type(numSet1)}, {numSet1}")
```

◆ Set 자료형

❖메서드 ➔ set객체변수명.메서드()

함수	기능		
intersection(집합변수)	교집합	S1 & S2	S1.intersection(S2)
union(집합변수)	합집합	S1 | S2	S1.union(S2)
difference(집합변수)	차집합	S1 – S2	S1.difference(S2)
add(값)	1개 값 추가		S1.add(4)
update(여러 개 값)	여러 개 한꺼번에 추가		S1.update([1,2,3])
remove(값)	특정 값 제어		S1.remove(값)

PYTHON 자료형 - DICT

PYTHON 자료형

◆ 맵핑 자료형

Dictionary	dic타입	class 'dict'	다양한 데이터

KEY	VALUE
1	홍길동
2	이나영
3	송혜교
4	홍길동
5	톰

◆ Dict 자료형

- **Key와 Value를 쌍**으로 구성
- Key를 통해서 Value을 얻는 방식
- 연관 배열 또는 해시(Hash)라고 함
- 중괄호{ }로 묶고 key:value를 콤마(,)로 구분
- 순서가 의미 없음
- **주의!! Key 중복**, 중복 시 하나의 키 제외한 나머지 무시

변수명={ 'key1':'value1', 'key2':'value2', 'key3':'value3' }

◆ Dict 자료형

❖ 생성

```
dic ={}

dic1={"name":"Tom", "age":10, "birth":"1224"}

dic2={1:"Hi", 2:"Apple", "Num":1}

dic3={"a":[1,2,3], "b":123}

dic4={1:("A","B",123), 2:"Happy"}
```

◆ Dict 자료형

❖값 읽기 : **객체변수명[Key]**

```
print("₩n==== Dic객체 요소 값 읽기 ======")
print(f"dic1[1]      => {dic1['name']}")
print(f"dic5[1]      => {dic5[1]}")
print(f"dic6[(1,2)] => {dic6[(1,2)]}")
```

◆ Dict 자료형

❖변경 및 요소 추가 : **객체변수명[Key] = Value**

```
print("₩n==== Dic객체 요소 값 변경 & 추가 ======")
print(f"[전] dic1    => {dic1}")

dic1['name']="KKK"                    # 존재 키일 경우 '변경'
print(f"[후] dic1    => {dic1}")

dic1['location']="DAEGU"              # 미존재 키일 경우 '추가'
print(f"[후] dic1    => {dic1}")
```

◆ Dict 자료형

❖요소 삭제

```
print("₩n===== Dic객체 요소 삭제 ==========")
del dic1['name']
print(f"[후] dic1    => {dic1}")

del(dic1['location'])
print(f"[후] dic1    => {dic1}")
```

◆ Dict 자료형

❖메서드 : **dict 객체변수명.메서드명()**

함수	기능
keys()	딕셔너리의 Key 리스트 리턴 => 2.7까지 딕셔너리의 **dict_keys 객체** 리턴 => 3.0 이후
values()	딕셔너리의 Value 리스트 리턴 => 2.7까지 딕셔너리의 **dict_values 객체** 리턴 => 3.0 이후
items()	딕셔너리의 Key-Value 쌍 튜플로 묶은 dict_items 객체 리턴
clear()	모든 쌍 지우기 즉, 딕셔너리안의 **모든 요소 삭제**
get(key)	Key로 Value 값 얻기 Key에 해당 값이 없으면 None 리턴
get(key , default)	Key에 해당 값이 없으면 기본값 리턴
key in 변수명	Key가 딕셔너리 안에 존재하는 지 체크 True / False 리턴

◆ Dict 자료형

❖ 메서드

```
mydic={"name":"Tom", "age":10, "birth":"1224"}

print("₩n======== Dic객체 메서드 ==============")
values=mydic.values()
print(f"values=> {type(values)}, {values}")

keys=mydic.keys()
print(f"keys=> {type(keys)}, {keys}")

items=mydic.items()
print(f"items=> {type(items)}, {items}")
```

◆ Dict 자료형

❖ 메서드

```
mydic={"name":"Tom", "age":10, "birth":"1224"}

print(f"mydic.get('name')=> {mydic.get('name')}")
print(f"mydic.get('location', '---')=> {mydic.get('location', '---')}")

print(f"[전] mydic=> {type(mydic)}, {mydic}")
mydic.update({'name':'Blue Sky', 'age':99, 'gender':'F'})
print(f"[후] mydic=> {type(mydic)}, {mydic}")

mydic.clear()
print(f"[후] mydic=> {type(mydic)}, {mydic}")
```